

El lenguaje C. El tipo de datos Puntero

puntero == dirección de memoria

Profesores Estructuras de Datos, 2026.

Dpto. Lenguajes y Ciencias de la Computación.

University of Málaga

Licenciado bajo [CC BY-NC 4.0](#)

```
#include <stdio.h>
int suma (int sumando1, int sumando2){ //Parámetros formales (parámetros)
    int resultado = sumando1+sumando2;
    printf("En procedimiento: Sumamos el valor %i a %i y da como resultado %i \n",sumando1,sumando2, resultado);
    return sumando1+sumando2;
}

void divisionEntera (int dividendo, int divisor, int cociente, int resto){
    cociente = dividendo/divisor;
    resto = dividendo%divisor;

    printf("En procedimiento: Si dividimos el valor %i entre %i y da como cociente %i y como resto de %i\n",dividendo,divisor, cociente, resto);
}

int main(){
    int numero1 = 6;
    int numero2 = 4;
    int resultado = suma (numero1,numero2); //Parámetros reales (argumentos)
    printf("En Main: Sumamos el valor %i a %i y da como resultado %i \n",numero1,numero2,resultado);

    int cociente, resto;

    divisionEntera(numero1,numero2,cociente,resto);
    printf("En Main: Si dividimos el valor %i entre %i y da como cociente %i y como resto de %i\n",numero1,numero2, cociente, resto);

    return 0;
}
```

¿Cómo accedo a memoria?

Operador	Nombre técnico	Función principal	Ejemplo de uso	Resultado / Qué devuelve
<code>&</code>	Operador de dirección (Address-of operator)	Obtiene la dirección de memoria física donde se encuentra almacenada una variable.	<pre>int *p = &variable;</pre>	La posición de memoria (un puntero) que ocupa <code>variable</code> .
<code>*</code>	Operador de indirección (Dereference operator)	Accede directamente al contenido o valor guardado en la dirección a la que apunta un puntero.	<pre>int valor = *p;</pre>	El dato real que está dentro de la dirección guardada en <code>p</code> .

Al direccionar (&p)...

- **Todas las direcciones miden lo mismo:** Ocupan 4 u 8 bytes en memoria (según si el sistema es de 32 o 64 bits).
- Guardar la dirección de un `char`, un `int` o un `double` es físicamente idéntico: solo almacena un número que indica una posición de memoria.

Al indireccionar (*p) ... ¡aquí entra el Tipo!

El tipo del puntero (`int*`, `char*`, `double*`) **no cambia la dirección**, pero le indica al compilador cómo actuar al usar `*p`:

- **Tamaño (¿Cuántos bytes leemos/escribimos?):**
 - `char*` → Lee/Escribe **1 byte**
 - `int*` → Lee/Escribe **4 bytes**
 - `double*` → Lee/Escribe **8 bytes**

Operador * sobrecargado

Permite declarar una variable de tipo puntero a un tipo:

```
int *ptr;
```

Permite acceder a una variable de tipo puntero ([indireccionar](#)):

```
*ptr = 1;
```

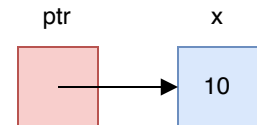
Declaración

- Declaración de una variable de tipo Puntero (tres formas diferentes)

```
int *ptr;  
int * ptr1;  
int* ptr2;
```

- Operador de dirección &
 - Devuelve la dirección en memoria de una variable
 - Es un operador unario

```
//ptr es una variable que contiene una dirección en la que se almacena un entero  
int *ptr;  
  
// x es una variable entera que está almacenada en memoria  
int x = 10;  
  
// ptr se le asigna la dirección de memoria dónde está almacenada x  
ptr = &x;
```



Declaración y uso

- Operador de indirección *
 - Devuelve el contenido de la zona de memoria a la que apunta un puntero
 - Es un operador unario
 - Es un operador sobrecargado (sirve para otras cosas). No confundir con la declaración de un tipo puntero o con el operador multiplicación

```
printf("valor = %d %d", x, *ptr);
```

Declaración y uso

```
// x es una variable que almacena un entero  
int x = 10;
```

```
//ptr es una variable que contiene una  
// dirección en la que se almacena un entero  
int *ptr;
```

```
ptr = &x;
```

```
//ptr2 es una variable que contiene una  
// dirección en la que se almacena un double  
double *ptr2;
```

x		1030
	10	1029
		1028
		1027
ptr		1026
	1029	1025
ptr2	??	1024



Ejercicio 1: Corrige este código

```
#include <stdio.h>

void divisionEntera (int dividendo, int divisor, int cociente, int resto){
    cociente = dividendo/divisor;
    resto = dividendo%divisor;

    printf("En procedimiento: Si dividimos el valor %i entre %i y da como cociente %i y como resto de %i\n",dividendo,divisor, cociente, resto);
}

int main(){
    int numero1 = 6;
    int numero2 = 4;
    int cociente, resto;

    divisionEntera(numero1,numero2,cociente,resto);
    printf("En Main: Si dividimos el valor %i entre %i y da como cociente %i y como resto de %i\n",numero1,numero2, cociente, resto);

    return 0;
}
```



Ejercicio 2

¿Se pueden ver los valores y las direcciones de memoria del cociente y el resto desde el depurador?

Ejercicio 3:

Arregla este código para que realice el intercambio

```
#include <stdio.h>

void swap1(int x, int y){
    int temp;
    temp = x;
    x=y;
    y=temp;
}

int main(void)
{

    int a,b;
    int c,d;
    a=c=4;
    b=d=6;

    swap1(a,b);

    //Usa el debug
    return 0;
}
```



Ejercicio 4

- Ejecuta este código (incluyelo dentro de un `main`)

```
int *ptrInt;    // contiene una dirección en la que se almacena un entero
char *ptrChar; // contiene una dirección en la que se almacena un carácter
float *ptrFloat; // contiene una dirección en la que se almacena un flotante

int a = 5;      // crea un entero, que está almacenado en la memoria del stack
char b = 'b';   // crea un carácter, que está almacenado en la memoria del stack
float f = 4.2f; // crea un entero, que está almacenado en la memoria del stack

ptrInt=&a;
ptrChar=&b;
ptrFloat=&f;

printf("Entero %llu, Char %llu, Float %llu\n", sizeof(a), sizeof(b), sizeof(f));
printf("Ptr Entero %llu, Ptr Char %llu, Ptr Float %llu\n", sizeof(ptrInt), sizeof(ptrChar), sizeof(ptrFloat));
printf("Entero %p, Char %p, Float %p\n", ptrInt, ptrChar, ptrFloat);
```

¿Ocupan diferente los tipo puntero (puntero entero, puntero char, puntero float)? ¿Por qué?

Ejemplo (0/3)

```
// * operador indirección (no confundir con tipos de variable)
// & operador dirección

// Un entero, 4 bytes
int x = 10;
// Un entero, 4 bytes
int y = 20;

// C trata los arrays como punteros al primer elemento,
// esto es como un int * que tendrá 3 enteros consecutivos, 12 bytes
int z[3];

// Variable puntero (dirección de memoria) a un entero
int *ptr;

//Obtengo la dirección de memoria dónde está x almacenado y lo asigno a la variable puntero ptr.
ptr = &x;

//indirección del puntero ptr, *ptr, voy a la memoria e interpreto los 4 bytes como un entero, se asigna a y.
y = *ptr;

//asigno el valor 0 a la indirección del puntero ptr, *ptr.
// Es decir, en la zona de memoria apuntada por ptr, almaceno el entero 0.
*ptr = 0;

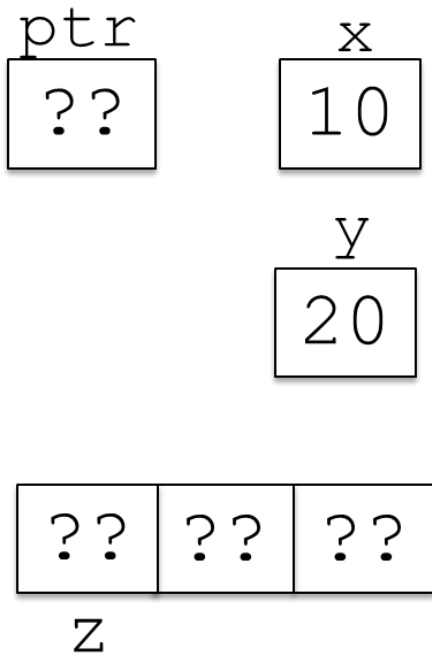
//la dirección de memoria de primer elemento del array, se asigna al puntero ptr (variable que almacena direcciones de memoria)
ptr = z;
```

¿Qué valor tiene x e y? ¿y ptr?

Ejemplo (1/3)

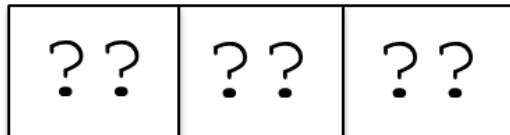
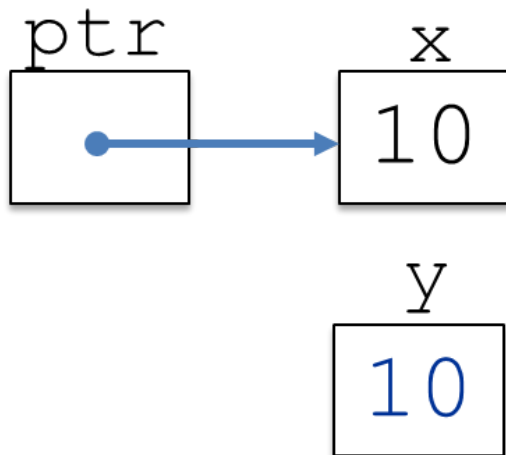
```
int x = 10;  
int y = 20;  
int z[3];
```

```
// ptr es una variable que almacena una dirección de memoria en la que se almacena/rá un entero  
// (es un puntero a un entero)  
int *ptr;
```



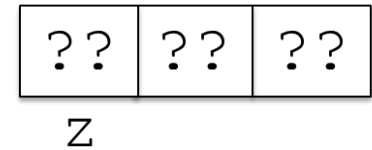
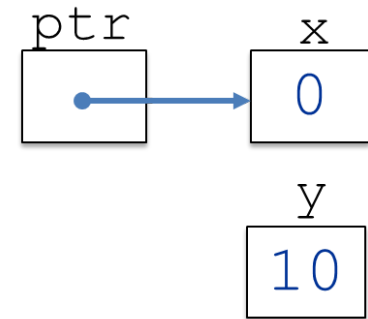
Ejemplo (2/3)

```
// ptr almacena la dirección de memoria dónde está almacenada la variable x
// (ptr apunta a x)
ptr = &x;
// Se desreferencia la dirección ptr para obtener su contenido y asignarlo a y, y vale 10
// (y se le asigna a dónde apunta ptr)
y = *ptr;
```

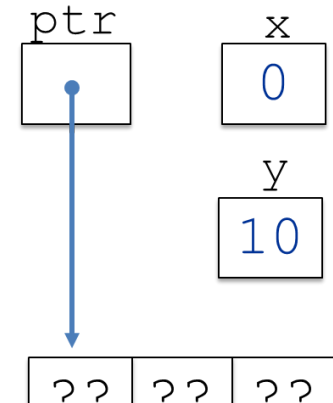


Ejemplo (3/3)

```
*ptr = 0; // Asigno 0 al valor al que referencia ptr, por tanto el valor de x es 0
```



```
ptr = &z[0]; // ptr almacena la dirección en la que se almacena z[0]. Apuntan al mismo sitio.
```



Ejemplo

```
// * operador indirección (no confundir con tipos de variable)
// & operador dirección
```

```
int x = 10;
int y = 20;
int z[3];
int *ptr;
```

```
ptr = &x;
y = *ptr;
*ptr = 0;
ptr = &z[0];
```

¿Qué valor tiene x e y? ¿y ptr?

x=0; y=10; ptr= dirección de memoria del primer elemento de z[0]

El Tipo Puntero y su utilidad en Procedimientos y Funciones

Procedimientos y Funciones

- C define funciones que pueden:
 - Devolver “nada” (void) -> Procedimientos (no se debe usar `return`)
 - Devolver valores de algún tipo (enteros, carácter, punteros, estructuras, etc)
- Argumentos de las funciones
 - Todos se **pasan por valor** pero....
 - **¿Es posible obtener una referencia?.**
 - En la llamada se debe proporcionar la dirección de la variable
 - En la definición formal el parámetro debe ser declarado como puntero a una variable del tipo correspondiente.

Parámetros por valor y referencia

¿INCORRECTO o CORRECTO?

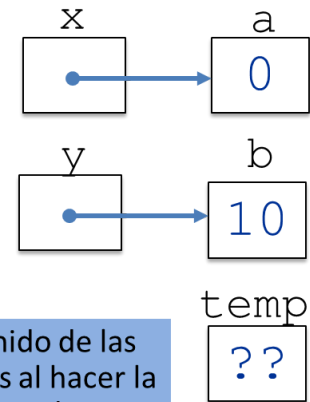
```
swap1 (a, b); //llamada
...
void swap1(int x, int y) {
    int temp;
    temp = x;
    x = y;
    y = temp;
}
```

¿INCORRECTO o CORRECTO?

```
swap2 (&a, &b); //llamada
...
void swap2(int* x, int* y) {
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

Paso de la referencia

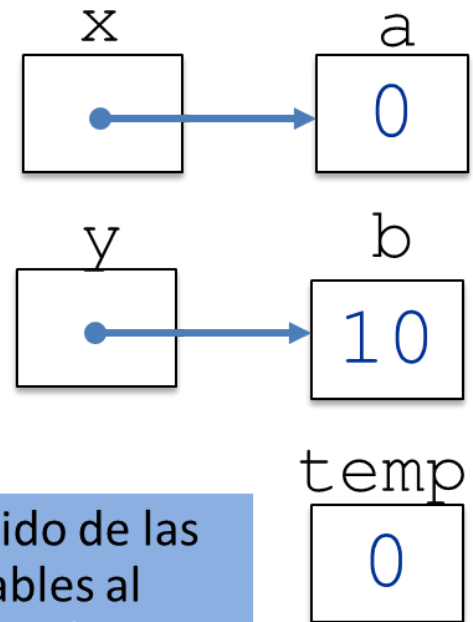
```
swap (&a, &b); //llamada
...
void swap(int* x, int* y) {
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```



Paso de la Referencia

```
swap (&a, &b); //llamada
...
void swap(int* x, int* y) {
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

← Línea 1

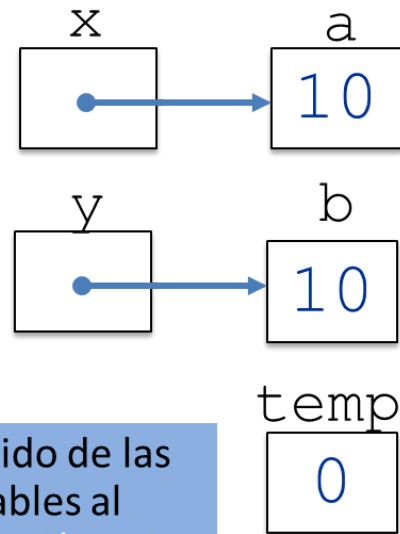


Contenido de las variables al ejecutar Línea 1

Paso de la Referencia

```
swap (&a, &b); //llamada
...
void swap(int* x, int* y) {
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

← Línea 2

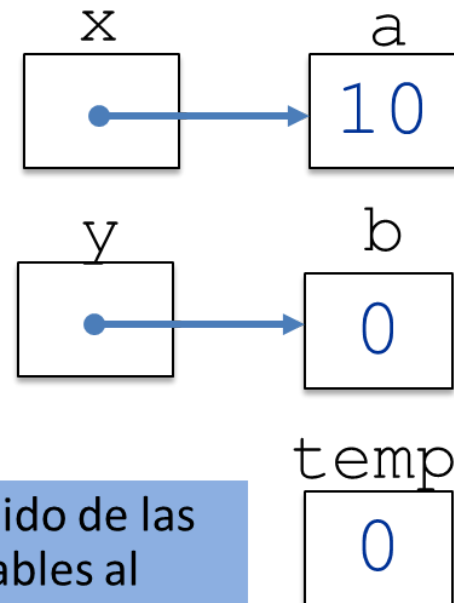


Contenido de las
variables al
ejecutar Línea 2

Paso de la Referencia

```
swap (&a, &b); //llamada
...
void swap(int* x, int* y) {
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

Línea 3



Contenido de las
variables al
ejecutar Línea 3

Paso de la Referencia

Paso por valor/ de la referencia

Realiza un programa que suma dos números y modifica uno de ellos por referencia.

1. Asignar dos números a dos variables en el `main`
2. Definir una función que suma dos números y modifica el primero de ellos para devolver la suma (paso por referencia).
3. Mostrar en el main los valores de las dos variables



El tipo Puntero

- Sabemos definir un tipo puntero
- Sabemos que el tipo puntero sirve para pasar parámetros por referencia
- Hemos comentado que una variable puntero puede apuntar a cualquier tipo
 - Solo hemos visto apuntando a tipos simples

El tipo Puntero

- Apuntar a tipos simples no aporta mucho con respecto a definir variables estáticas.
- Hay otras formas de aprovechar la flexibilidad de los Punteros, y para ello hay que usar el Montículo (Heap) que permite hacer memoria dinámica y las estructuras.

